1. The Artificial Neuron.

An artificial neuron is the basic unit of a neural network. It receives several numeric inputs and multiplies each input by a weight. A weight measures how strongly an input influences the neuron: a large positive weight amplifies the input, a negative weight suppresses it, and a weight near zero means the input is mostly ignored. The neuron adds the weighted inputs together and then adds a bias. The bias shifts the result up or down, so the neuron can produce an output even when all inputs are zero. The weighted sum plus the bias is often written as z = w1*x1 + w2*x2 + ... + wn*xn + b. Finally, the neuron passes z through an activation function. A common choice is the sigmoid function, sigmoid(z) = 1 / (1 + e^(-z)), which squashes any number into the range between 0 and 1. Large positive values of z give outputs close to 1, large negative values give outputs close to 0, and z = 0 gives exactly 0.5. Another common choice is ReLU, which outputs z when z is positive and 0 otherwise.

2. From Neurons to Networks.

A neural network connects many neurons in layers. The input layer holds the raw data, one or more hidden layers transform it, and the output layer produces the final result. In a feedforward network, information flows in one direction: from the inputs, through the hidden layers, to the outputs. This one-way flow is called the forward pass. During the forward pass, each neuron in a layer receives the outputs of every neuron in the previous layer, computes its own weighted sum plus bias, applies its activation function, and sends its output to the next layer. The strength of the signal along each connection is the product of a weight and the activation that feeds into it. Because every hidden neuron has its own weights, different neurons respond to different patterns in the input. Stacking layers lets a network combine simple patterns into more complex ones.

3. Measuring Mistakes: The Loss.

A network learns from examples whose correct answers, called targets, are known. For each example, the network makes a prediction, and a loss function measures how far the prediction is from the target. A common loss for numeric predictions is the mean squared error (MSE): the average of the squared differences between predictions and targets. Squaring makes every error positive and penalizes large errors more than small ones. A loss of zero means every prediction matches its target exactly. The goal of training is to find the weights that make the loss as small as possible. If we plot the loss against a single weight, the curve often looks like a valley, and the best weight sits at the bottom of the valley.

4. Learning by Gradient Descent.

Gradient descent is the most common way to reduce the loss. The gradient of the loss with respect to a weight tells us which direction increases the loss, and how steeply. To reduce the loss, we move the weight a small step in the opposite direction: new weight = old weight - learning rate x gradient. The learning rate controls the size of each step. If the learning rate is too small, training makes progress very slowly. If it is too large, each step can overshoot the bottom of the valley, and the loss may grow instead of shrink. Training repeats this update many times, and in deep networks an algorithm called backpropagation computes the gradients for every weight efficiently by applying the chain rule from the output layer back toward the input layer. Over many steps, the weights settle near values that make the predictions match the targets closely.